DOCKER VOLUME

Docker Volume is a persistent storage mechanism used to store and manage data generated by Docker containers. Data stored in volumes remains available even if the container is stopped, removed, or recreated.

• Stores container data permanently outside the container.

• Data remains available even if the container is stopped or deleted.

• Provides persistent storage for Docker applications.

• Isolated from the container lifecycle.

• Commonly used for databases, application files, and backups

Docker Volume Uses

• **Data Persistence** - Stores container data permanently even after container deletion.

• **Data Sharing** - Allows multiple containers to access and share the same data.

• **Backup and Recovery** - Makes it easier to back up and restore application data.

• **Database Storage** - Commonly used to store database files and application data.

• **Container Independence** - Data remains separate from the container lifecycle.

WITH VOLUME VS WITHOUT VOLUME

• **With Volume** → Container deleted, data remains safe.

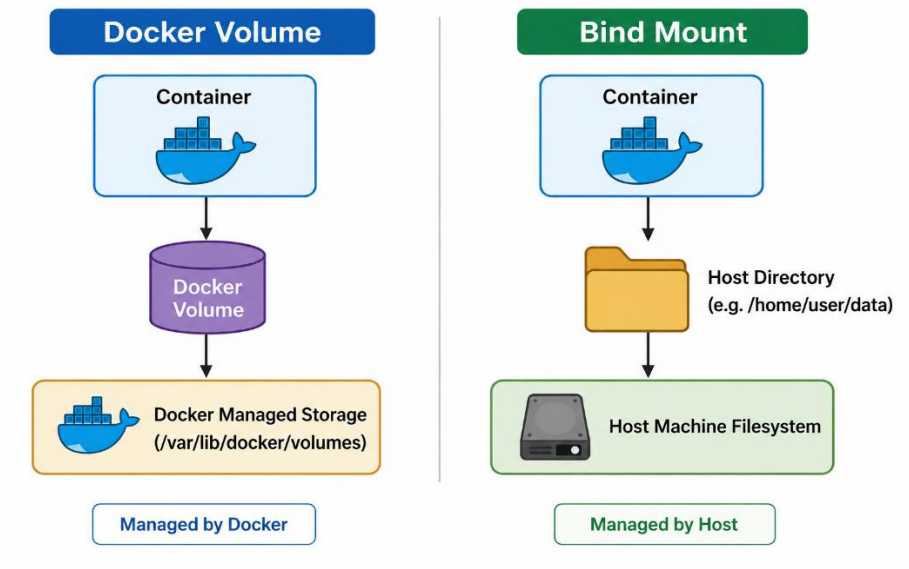
• **Without Volume** → Container deleted, data is also deleted.

• **With Volume** → Data can be reused by new containers.

• **Without Volume** → Data is lost when the container is removed.

Docker Volume vs Bind Mount

Docker Volume	**Bind Mount**
Managed by Docker	Managed by the Host OS
Stored in Docker's default location (/var/lib/docker/volumes)	Stored in any directory on the host machine



DOCKER NETWORK

Docker Network is a communication layer that enables Docker containers to communicate with each other, the host machine, and external networks.

Docker Network Uses

• **Container Communication** - Allows containers to communicate with each other.
• **Application Connectivity** - Connects multiple containers in multi-container applications.
• **Isolation** - Separates container traffic from other networks.
• **External Access** - Allows containers to communicate with external systems and the internet.
• **Microservices Support** - Enables communication between different application services.

Common Docker Network Types

• **Bridge Network** - Default network used for communication between containers on the same host.
• **Host Network** - Container shares the host machine's network.

HANDS-ON FOR DOCKER VOLUME AND DOCKER NETWORKS

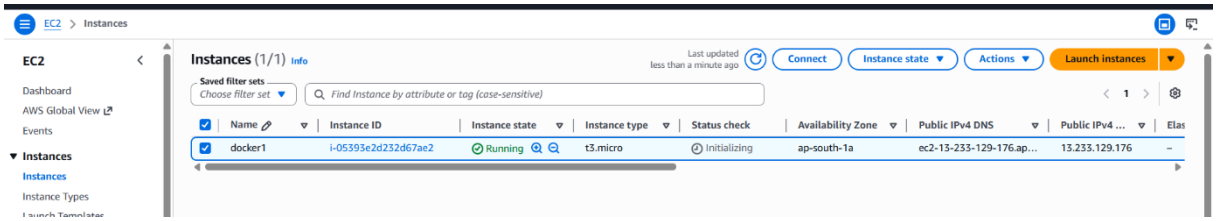
DOCKER VOLUME

Screenshot: Create Ubuntu EC2 Instance

• Launch a new Ubuntu EC2 instance in AWS

• Configure instance settings and security group

• Ubuntu instance created successfully and in running state



Screenshot: Connect to Ubuntu Instance and Update Packages

• Connect to the Ubuntu EC2 instance using SSH

• Execute sudo su

• Execute apt update -y

• System packages updated successfully

```
ubuntu@ip-172-31-43-1:~$ sudo su
root@ip-172-31-43-1:/home/ubuntu# apt update
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [136 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 Packages [1483 kB]
```

Screenshot: Install Docker on Ubuntu

• Execute apt install docker.io -y

• Docker package downloaded and installed successfully on the Ubuntu instance

```
root@ip-172-31-40-145:/home/ubuntu# apt install docker.io
Installing:
  docker.io

Installing dependencies:
  bridge-utils  containerd  dns-root-data  dnsmasq-base  pigz  runc  ubuntu-fan

Suggested packages:
  ifupdown  aufs-tools  cgroupfs-mount  | cgroup-lite  debootstrap  docker-buildx  docker-compose-v2  docker-doc  rinse  rootlesskit  zfs-fuse  | zfsutils

Summary:
  Upgrading: 0, Installing: 8, Removing: 0, Not Upgrading: 39
  Download size: 74.0 MB
  Space needed: 281 MB / 4685 MB available
```

Screenshot: Start Docker Service and Check Status

• Execute systemctl start docker

• Execute systemctl enable docker

• Execute systemctl status docker

```
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# systemctl start docker
root@ip-172-31-40-145:/home/ubuntu# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-05-30 15:40:19 UTC; 2min 28s ago
 Invocation: 3de11a3c09464c50b002e5ab9751e643
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 2621 (dockerd)
      Tasks: 9
     Memory: 26.8M (peak: 27.8M)
        CPU: 354ms
     CGroup: /system.slice/docker.service
             └─2621 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Screenshot: Docker Volume Commands Displayed

- sudo su
- docker volumes
- Docker volumes Commands Displayed

```
root@ip-172-31-40-145:/home/ubuntu# docker volume
Usage:  docker volume COMMAND

Manage volumes

Commands:
  create      Create a volume
  inspect     Display detailed information on one or more volumes
  ls          List volumes
  prune       Remove unused local volumes
  rm          Remove one or more volumes
```

Screenshot: Docker Volume Creation and Listing

- docker volume ls (no volumes shown initially)
- docker volume create myvolume
- docker volume ls (myvolume is created and shown)
- myvolume is displayed in the list successfully

```
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker volume ls
DRIVER    VOLUME NAME
root@ip-172-31-40-145:/home/ubuntu# docker volume create myvolume
myvolume
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker volume ls
DRIVER    VOLUME NAME
local     myvolume
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Pull HTTPD Docker Image

- docker pull httpd
- HTTPD image downloaded successfully from Docker Hub.

```
root@ip-172-31-40-145:/home/ubuntu# docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
ed804d184361: Pull complete
```

Screenshot: List Docker Images

- docker images
- All downloaded Docker images are displayed successfully.

```
root@ip-172-31-40-145:/home/ubuntu# docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
httpd:latest   2a7deaaaf357        177MB         47.6MB
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Run HTTPD Container with Volume

- docker run -itd --name con1 -p 8080:80 -v myvolume:/app/data httpd:latest
- HTTPD container created and started successfully
- Port 8080 mapped to 80
- Volume myvolume attached to container data directory

```
root@ip-172-31-40-145:/home/ubuntu# docker run -itd --name con1 -p 8080:80 -v myvolume:/app/data httpd:latest
f6d5c4a72da4a793c96bda87ce6bd7fed6bb69e85d5e2f4768f5e4bd1a666a3e
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: List Running Containers and Enter Running Container

- docker ps
- Running containers are displayed successfully.
- docker exec -it <httpd container id> /bin/bash
- Entered into the running HTTPD container successfully.

```
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND              CREATED              STATUS              PORTS                                       NAMES
f6d5c4a72da4   httpd:latest   "httpd-foreground"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   con1
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker exec -it f6d5c4a72da4 /bin/bash
root@f6d5c4a72da4:/usr/local/apache2#
```

Screenshot: Create Files Inside Container Volume

- cd /app/data (navigate to volume-mounted directory inside container)
- touch file1 file2 file3 (create sample files inside container)
- ls (verify files are created and displayed)
- exit (exit from container shell successfully)

```
root@f6d5c4a72da4:/usr/local/apache2# cd /app/data/
root@f6d5c4a72da4:/app/data#
root@f6d5c4a72da4:/app/data# touch file1 file2 file3
root@f6d5c4a72da4:/app/data#
root@f6d5c4a72da4:/app/data# ls
file1  file2  file3
root@f6d5c4a72da4:/app/data# exit
exit
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Docker Volume Directory Check

- cd /var/lib/docker/volumes (navigate to Docker volume storage location)
- ls (list all volumes on host machine)
- myvolume is shown successfully in the directory

```
root@ip-172-31-40-145:/home/ubuntu# cd /var/lib/docker/volumes/
root@ip-172-31-40-145:/var/lib/docker/volumes#
root@ip-172-31-40-145:/var/lib/docker/volumes# ls
backingFsBlockDev  metadata.db  myvolume
```

Screenshot: Verify Volume Data in Host Machine

- cd myvolume (enter the created volume directory)
- ls (view volume structure)
- cd _data (navigate to actual stored data folder)
- ls (files created inside container are displayed successfully on host machine)

```
root@ip-172-31-40-145:/var/lib/docker/volumes# cd myvolume/
root@ip-172-31-40-145:/var/lib/docker/volumes/myvolume#
root@ip-172-31-40-145:/var/lib/docker/volumes/myvolume# ls
_data
root@ip-172-31-40-145:/var/lib/docker/volumes/myvolume# cd _data/
root@ip-172-31-40-145:/var/lib/docker/volumes/myvolume/_data# ls
file1  file2  file3
root@ip-172-31-40-145:/var/lib/docker/volumes/myvolume/_data#
```

BINDMOUNTS

Screenshot: Pull Nginx Docker Image

- docker pull nginx
- Nginx image downloaded successfully from Docker Hub.

```
root@ip-172-31-40-145:/home/ubuntu# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
13fd728be9eb: Pull complete
830625e1ac85: Pull complete
b4a248c845e5: Pull complete
7f8b1a2b17d8: Pull complete
```

Screenshot: Create and Navigate Bind Mount Directory

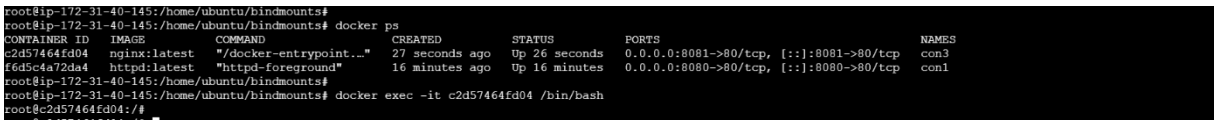
- mkdir bindmounts
- cd bindmounts
- Bind mount directory created and accessed successfully.

Screenshot: Run Nginx Container with Bind Mount

- pwd (shows /home/ubuntu/bindmounts)
- docker run -itd --name con3 -p 8081:80 -v /home/ubuntu/bindmounts:/app/data nginx:latest
- Nginx container started successfully
- Bind mount attached between host and container

Screenshot: Access Nginx Container Shell

- docker ps (nginx container is running and shown in list)
- docker exec -it <nginx container id> /bin/bash
- Entered into the running Nginx container successfully



Screenshot: Create Files in Bind Mount Directory

- cd /app/data (navigate to bind mount directory inside container)
- touch file4 file5 (create new files in container)
- ls (file4 and file5 are displayed successfully)
- exit (exit from container shell)

```
root@c2d57464fd04:/# cd /app/data
root@c2d57464fd04:/app/data# touch file4 file5
root@c2d57464fd04:/app/data# ls
file4  file5
root@c2d57464fd04:/app/data# exit
```

Screenshot: Verify Bind Mount on Host Machine

- ls
- Files created inside container are visible on host machine successfully (file4, file5 shown)

```
root@ip-172-31-40-145:/home/ubuntu/bindmounts# ls
file4  file5
root@ip-172-31-40-145:/home/ubuntu/bindmounts#
```

DOCKER NETWORK

Screenshot: Docker Network Commands Displayed

- sudo su
- docker network
- Docker Network Commands Displayed

```
ubuntu@ip-172-31-40-145:~$ sudo su
root@ip-172-31-40-145:/home/ubuntu# docker network
Usage:  docker network COMMAND

Manage networks

Commands:
  connect     Connect a container to a network
  create      Create a network
  disconnect  Disconnect a container from a network
  inspect     Display detailed information on one or more networks
  ls          List networks
  prune       Remove all unused networks
  rm          Remove one or more networks

Run 'docker network COMMAND --help' for more information on a command.
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Docker Network List

- docker network ls
- Default Docker networks (bridge, host, none) are displayed successfully.

```
root@ip-172-31-40-145:/home/ubuntu# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
3a44e41d9cac   bridge    bridge    local
20353d5cfb1f   host      host      local
20a5ea4ab2fd   none      null      local
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Create Docker Network

- docker network create mynetwork
- Custom Docker network created successfully.

```
root@ip-172-31-40-145:/home/ubuntu# docker network create mynetwork
88100552a69377be069d259b97461e845346f95ff7de57e25b8b9c53f6b249f2
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Verify Docker Network Creation

- docker network ls
- Default networks and mynetwork are displayed successfully.

```
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker network ls
NETWORK ID     NAME        DRIVER    SCOPE
3a44e41d9cac   bridge      bridge    local
20353d5cfb1f   host        host      local
88100552a693   mynetwork   bridge    local
20a5ea4ab2fd   none        null      local
root@ip-172-31-40-145:/home/ubuntu#
```

Screenshot: Connect Container to Docker Network

- docker network connect mynetwork <nginx container id>
- Nginx container connected to mynetwork successfully.

```
root@ip-172-31-40-145:/home/ubuntu# docker network connect mynetwork c2d57464fd04
```

Screenshot: Inspect Nginx Container Network

- docker inspect <nginx container id>
- Container details displayed successfully
- mynetwork is shown under network settings
- Network connection verified successfully.

```
root@ip-172-31-40-145:/home/ubuntu# docker inspect c2d57464fd04
[
```

```
        },
        "mynetwork": {
            "IPAMConfig": {
                "IPv4Address": "",
                "IPv6Address": ""
            },
            "Links": null,
            "Aliases": [],
            "DriverOpts": {},
            "GwPriority": 0,
            "NetworkID": "88100552a69377be069d259b97461e845346f95ff7de57e25b8b9c53f6b249f2",
            "EndpointID": "e0fe2829d6d093adfba16d4899eddaeee7f15a88a1ee4c6bb54b51a74a395773",
            "Gateway": "172.18.0.1",
            "IPAddress": "172.18.0.2",
            "MacAddress": "02:c1:e4:25:31:c3",
            "IPPrefixLen": 16,
            "IPv6Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "DNSNames": [
                "con3",
                "c2d57464fd04"
```


Screenshot: Copy Container IP Address

- docker inspect <nginx container id>
- Locate the container IP address under network settings
- Copy the IP address
- Paste and save it in Notepad successfully


```
"Networks": {
    "bridge": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": null,
        "DriverOpts": null,
        "GwPriority": 0,
        "NetworkID": "3a44e41d9cacf32c200dcddb772e4d4d1f295164074f54c0fb8963f4b30c6bdb",
        "EndpointID": "d1962f79791f5d2c38c2d8cbed6f8475e2527a7d36b0f012c00fdd6215c6cce0",
        "Gateway": "172.17.0.1",
        "IPAddress": "172.17.0.3",
        "MacAddress": "36:06:f0:5b:36:b8",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
```


Screenshot: Connect HTTPD Container to Docker Network

- docker network connect mynetwork <httpd container id>
- HTTPD container connected to mynetwork successfully.

```
root@ip-172-31-43-1:/home/ubuntu#
root@ip-172-31-43-1:/home/ubuntu# docker network connect mynetwork f6d5c4a72da4
```

Screenshot: Inspect HTTPD Container Network

- docker inspect <httpd container id>
- Container details displayed successfully

- mynetwork is shown under network settings
- Network connection verified successfully.

```
root@ip-172-31-43-1:/home/ubuntu#
root@ip-172-31-43-1:/home/ubuntu# docker inspect f6d5c4a72da4
```

Screenshot: Copy HTTPD Container IP Address

- docker inspect <httpd container id>
- Locate the container IP address under network settings
- Copy the IP address
- Paste and save it in Notepad successfully


```
    },
    "Networks": {
        "bridge": {
            "IPAMConfig": null,
            "Links": null,
            "Aliases": null,
            "DriverOpts": null,
            "GwPriority": 0,
            "NetworkID": "3a44e41d9cacf32c200dcddb772e4d4d1f295164074f54c0fb8963f4b30c6bdb",
            "EndpointID": "175b43337605e747e4ab77679720ee5657e493fc0aa0e2d07ad743034dc33e98",
            "Gateway": "172.17.0.1",
            "IPAddress": "172.17.0.2",
            "MacAddress": "76:90:1c:3a:cc:ac",
            "IPPrefixLen": 16,
            "IPv6Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "DNSNames": null
```


Screenshot: Enter Nginx Container

- docker exec -it <nginx container id> /bin/bash
- Entered into the running Nginx container successfully.

```
root@ip-172-31-40-145:/home/ubuntu#
root@ip-172-31-40-145:/home/ubuntu# docker exec -it c2d57464fd04 /bin/bash
root@c2d57464fd04:/#
```

Screenshot: Install Ping in Nginx Container

- apt update
- apt install iputils-ping -y
- Ping utility installed successfully inside the container.

```
root@c2d57464fd04:/#  apt update
apt install iputils-ping -y
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [176 kB]
Fetched 10.1 MB in 1s (8900 kB/s)
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  iputils-ping

Installing dependencies:
  linux-sysctl-defaults
```

Screenshot: Ping HTTPD Container IP from Nginx Container

- ping <httpd container IP>
- Successful ping response received
- Network communication between containers verified successfully.

```
root@c2d57464fd04:/# ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.094 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.050 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.053 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.048 ms
64 bytes from 172.17.0.2: icmp_seq=5 ttl=64 time=0.053 ms
64 bytes from 172.17.0.2: icmp_seq=6 ttl=64 time=0.053 ms
64 bytes from 172.17.0.2: icmp_seq=7 ttl=64 time=0.051 ms
```

Screenshot: Exit Nginx and Enter HTTPD Container

- exit (exit from Nginx container)
- docker exec -it <httpd container id> /bin/bash
- Entered into HTTPD container successfully

```
root@ip-172-31-40-145:/home/ubuntu# docker exec -it f6d5c4a72da4 /bin/bash
root@f6d5c4a72da4:/usr/local/apache2#
```

Screenshot: Install Ping in HTTPD Container

- apt update
- apt install iputils-ping -y
- Ping utility installed successfully inside HTTPD container.

```
root@f6d5c4a72da4:/usr/local/apache2# apt update
apt install iputils-ping -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [176 kB]
Fetched 9943 kB in 1s (9154 kB/s)
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  iputils-ping

Installing dependencies:
  linux-sysctl-defaults
```

Screenshot: Ping Nginx Container IP from HTTPD Container

- ping <nginx container IP>
- Successful ping response received
- Communication between HTTPD and Nginx containers verified successfully

```
root@f6d5c4a72da4:/usr/local/apache2# ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
64 bytes from 172.17.0.3: icmp_seq=1 ttl=64 time=0.067 ms
64 bytes from 172.17.0.3: icmp_seq=2 ttl=64 time=0.047 ms
64 bytes from 172.17.0.3: icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from 172.17.0.3: icmp_seq=4 ttl=64 time=0.053 ms
64 bytes from 172.17.0.3: icmp_seq=5 ttl=64 time=0.048 ms
64 bytes from 172.17.0.3: icmp_seq=6 ttl=64 time=0.049 ms
64 bytes from 172.17.0.3: icmp_seq=7 ttl=64 time=0.053 ms
64 bytes from 172.17.0.3: icmp_seq=8 ttl=64 time=0.052 ms
```

Docker Multi-Network Container Communication

Screenshot: Pull Ubuntu Image

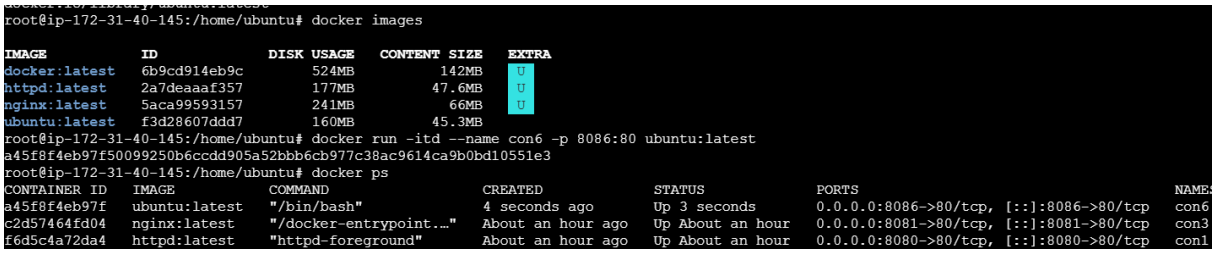
- docker pull ubuntu
- Ubuntu image downloaded successfully

```
root@ip-172-31-40-145:/home/ubuntu# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
1c24335ddd46: Pull complete
6f5c5aa4e145: Pull complete
9bcf140d7f0f: Download complete
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64
Status: Downloaded newer image for ubuntu:latest
```

Screenshot: List Images, Run Container, and Verify

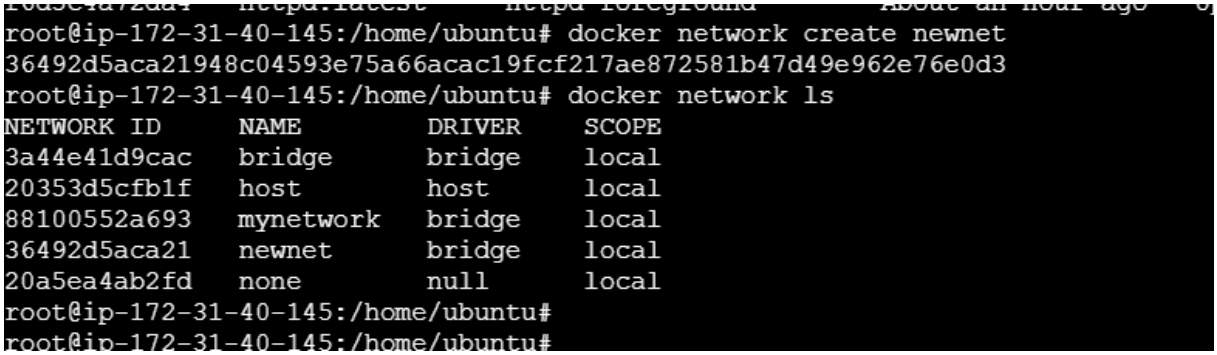
- docker images
- Ubuntu, HTTPD, and Nginx images are displayed successfully
- docker run -itd --name con5 -p 8084:80 nginx:latest

- Nginx container started successfully
- docker ps
- Running containers are displayed successfully, con5 is active



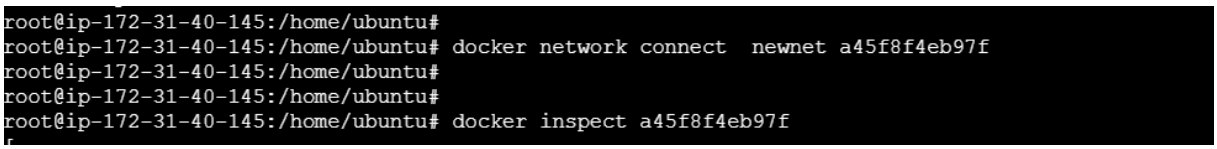
Screenshot: Create and Verify Docker Network

- docker network create newnet
- New Docker network created successfully
- docker network ls
- All networks including newnet are displayed successfully



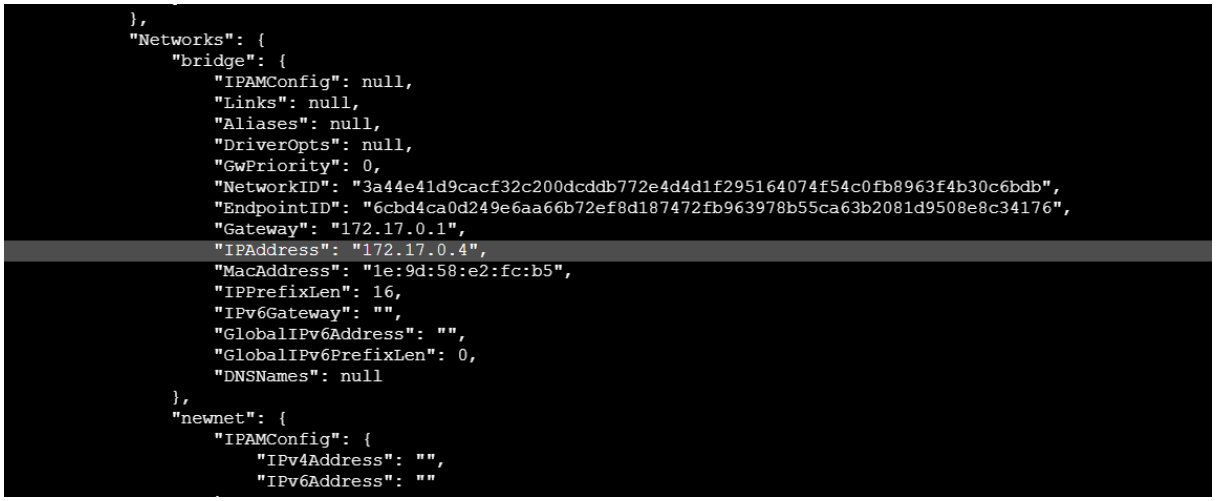
Screenshot: Connect Ubuntu Container and Inspect Network

- docker network connect newnet <ubuntu container id>
- Ubuntu container connected to newnet successfully
- docker inspect <ubuntu container id>
- Container details displayed successfully
- Network newnet is shown under network settings
- IP address and network configuration verified successfully



Screenshot: Copy Ubuntu Container IP Address

- docker inspect <ubuntu container id>
- Locate the container IP address under network settings
- Copy the IP address
- Paste and save it in Notepad successfully

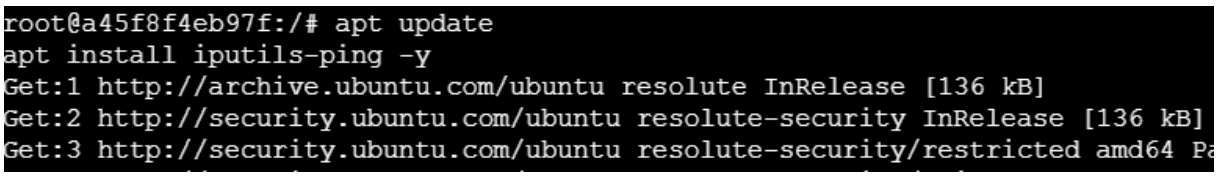


Screenshot: Enter Ubuntu Container

- docker exec -it <ubuntu container id> /bin/bash
- Entered into the Ubuntu container successfully

Screenshot: Install Ping in Ubuntu Container

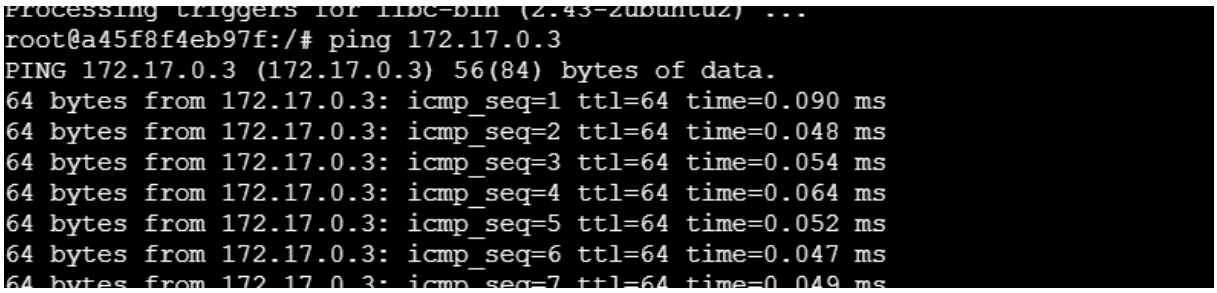
- apt update
- apt install iputils-ping -y
- Ping utility installed successfully inside Ubuntu container.



Screenshot: Ping Between Containers on Different Networks

- ping <httpd container IP>
- Ping request sent from Ubuntu container (newnet)

- HTTPD container is in mynetwork
- Ping successful and replies received
- Containers are able to communicate even across different networks (due to Docker network connectivity / shared bridge behavior or multiple network attachment)

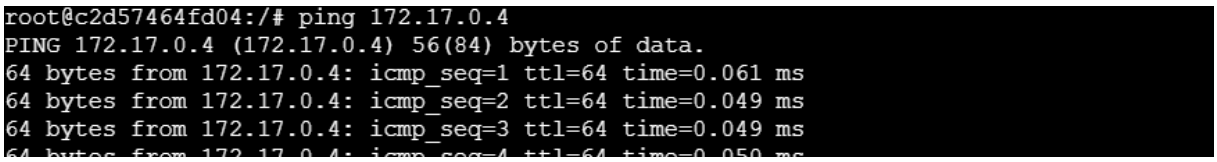


Screenshot: Enter HTTPD Container

- docker exec -it <httpd container id> /bin/bash
- Entered into the HTTPD container successfully

Screenshot: Ping Ubuntu Container IP from HTTPD Container

- ping <ubuntu container IP>
- Ping request sent successfully
- Communication between containers on different networks tested successfully



MULTISTAGE DOCKERFILE

Multi-Stage Dockerfile is a Dockerfile that uses multiple FROM instructions to create images in different stages. It helps reduce the final image size by copying only the required files from one stage to another.

Uses of Multi-Stage Dockerfile

• Reduces Docker image size.

• Removes unnecessary build tools and dependencies from the final image.

• Improves application security.

• Creates cleaner and optimized Docker images.

• Separates build and runtime environments.

Features of Multi-Stage Dockerfile

• Uses multiple FROM instructions.

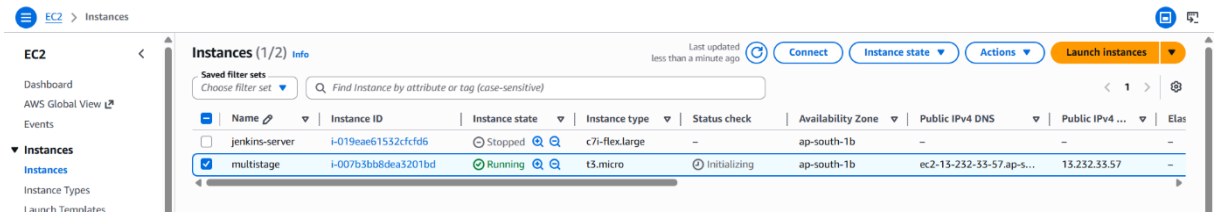
• Creates lightweight production images.

• Improves image build efficiency.

• Reduces storage and network usage.

• Simplifies Dockerfile management.

HANDS-ON




```
[ec2-user@ip-172-31-2-29 ~]$ sudo su
[root@ip-172-31-2-29 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Tue Jun  2 07:14:24 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-2-29 ec2-user]# yum install docker -y
Last metadata expiration check: 0:00:17 ago on Tue Jun  2 07:14:24 2026.
Dependencies resolved.
================================================================================
 Package                    Architecture                Version
================================================================================
Installing:
 docker                     x86_64                      25.0.14-1.amzn2023.0.6
```

```
[root@ip-172-31-2-29 ec2-user]# systemctl start docker
[root@ip-172-31-2-29 ec2-user]# systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[root@ip-172-31-2-29 ec2-user]# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-06-02 07:15:30 UTC; 17s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 20549 (dockerd)
      Tasks: 9
```

```
[root@ip-172-31-38-104 ec2-user]# git clone https://github.com/Reshufowzi/java-multistage-app.git
Cloning into 'java-multistage-app'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 14 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), done.
Resolving deltas: 100% (1/1), done.
[root@ip-172-31-38-104 ec2-user]# ls
java-multistage-app
[root@ip-172-31-38-104 ec2-user]# cd java-multistage-app/
[root@ip-172-31-38-104 java-multistage-app]#
```

```
[root@ip-172-31-38-104 java-multistage-app]#
[root@ip-172-31-38-104 java-multistage-app]# vi Dockerfile
```

With Multistage Dockerfile

```
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app
COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

# -------- Run Stage --------
FROM eclipse-temurin:17-jdk-jammy

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

CMD ["java", "-jar", "app.jar"]
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --
```

```
i-007b3bb8dea3201bd (multistage)
PublicIPs: 13.232.33.57  PrivateIPs: 172.31.2.29
CloudShell  Feedback  Console Mobile App
```

```
[root@ip-172-31-38-104 java-multistage-app]#
[root@ip-172-31-38-104 java-multistage-app]# docker build -t image .
[+] Building 4.4s (5/13)
 => [internal] load build definition from Dockerfile
[+] Building 4.5s (5/13)
 => [internal] load build definition from Dockerfile
[+] Building 4.6s (5/13)
```

```
[root@ip-172-31-38-104 java-multistage-app]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED              SIZE
image        latest    95601840623e   About a minute ago   435MB
[root@ip-172-31-38-104 java-multistage-app]#
```

```
[root@ip-172-31-38-104 java-multistage-app]#
[root@ip-172-31-38-104 java-multistage-app]# vi Dockerfile
```

Without Multistage Dockerfile

Size of the image differs for with multistage file and without multistage file

```
[root@ip-172-31-38-104 java-multistage-app]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
image        latest    95601840623e   3 minutes ago    435MB
image1       latest    57c326a5a597   3 minutes ago    547MB
[root@ip-172-31-38-104 java-multistage-app]#
```